

Plataforma de Serviços de Rede Baseada em containers

Daniel Taveira - a44319

Trabalho realizado sob a orientação de

Prof. Nuno Rodrigues

Prof. Eduardo Costa

Licenciatura em Engenharia Informática

2025-2026

Plataforma de Serviços de Rede Baseada em containers

Relatório da UC de Projeto
Licenciatura em Engenharia Informática
Escola Superior de Tecnologia e Gestão

Daniel Taveira - a44319

2025-2026

A Escola Superior de Tecnologia e de Gestão não se responsabiliza pelas opiniões expressas neste relatório.

Declaro que o trabalho descrito neste relatório é da minha autoria e é da minha vontade que o mesmo seja submetido a avaliação.

Daniel Taveira - a44319

Dedicatória

(Facultativo) Dedico este trabalho a ...

Agradecimentos

(Facultativo) Agradeço a ...

Resumo

O presente projeto tem como objetivo principal a conceção e implementação de uma plataforma de serviços de rede gerida de forma reprodutível, recorrendo exclusivamente a ferramentas *open source*. A infraestrutura desenvolvida engloba serviços fundamentais para uma rede de pequena a média dimensão, tais como servidor DNS, *Reverse Proxy* HTTPS, serviços *web*, e-mail, servidor de ficheiros e VPN.

Em contraste com abordagens monolíticas tradicionais, todos os serviços são implementados em *containers* (Docker/Podman) através de configuração declarativa, garantindo o versionamento e a reprodutibilidade do ambiente. O trabalho foca-se na aplicação rigorosa de boas práticas de segurança, incluindo o princípio do mínimo privilégio, *hardening* de serviços e encriptação TLS. Adicionalmente, o projeto integra mecanismos avançados de observabilidade (para métricas e *logs* centralizados) e a orquestração de *deployments* através de ferramentas como Ansible e *pipelines* de CI/CD. O resultado final demonstra como um pequeno *data center* pode ser gerido de forma automatizada, ágil e resiliente a falhas.

Palavras-chave: *Containers*, Orquestração, Serviços de Rede, CI/CD, Observabilidade, *Open Source*.

Abstract

The main objective of this project is the design and implementation of a network services platform managed in a reproducible way, using exclusively open-source tools. The developed infrastructure encompasses fundamental services for a small to medium-sized network, such as a DNS server, HTTPS Reverse Proxy, web services, e-mail, file server, and a VPN.

In contrast to traditional monolithic approaches, all services are implemented in containers (Docker/Podman) through declarative configuration, ensuring environment versioning and reproducibility. The work focuses on the strict application of security best practices, including the principle of least privilege, service hardening, and TLS encryption. Additionally, the project integrates advanced observability mechanisms (for metrics and centralized logs) and deployment orchestration using tools like Ansible and CI/CD pipelines. The final result demonstrates how a small data center can be managed in an automated, agile, and fault-resilient manner.

Keywords: Containers, Orchestration, Network Services, CI/CD, Observability, Open Source.

Índice

1	Introdução	1
1.1	Contextualização e Motivação	1
1.2	Objetivos	2
1.3	Metodologia	3
1.4	Estrutura do Documento	3
2	Estado da Arte	5
2.1	Virtualização e <i>Containers</i> : Evolução e Diferenças	5
2.2	Docker e Docker Compose	6
2.2.1	Podman como Alternativa <i>Rootless</i>	7
2.3	Infraestrutura como Código (IaC) e Ansible	7
2.4	Serviços de Rede em <i>Containers</i>	8
2.4.1	DNS com Pi-hole	8
2.4.2	<i>Reverse Proxy</i> com Nginx	9
2.4.3	VPN com WireGuard	9
2.5	Observabilidade e Monitorização	10
2.6	CI/CD e GitLab	11
3	Levantamento das Necessidades Funcionais	13
3.1	Serviços de Rede a Disponibilizar	13
3.2	Requisitos Técnicos e de Arquitetura	15
3.3	Ambiente Base: Virtualização e Sistema Operativo	15

3.3.1	Configuração de Rede do Servidor	16
3.3.2	Acesso Externo: Router em modo DMZ	16
4	Arquitetura e Implementação	19
4.1	Arquitetura Geral da Plataforma	19
4.2	Configuração do Ambiente Base (Ubuntu Server)	20
4.2.1	Instalação e Configuração Inicial	20
4.2.2	Configuração de IP Estático com Netplan	20
4.2.3	Problema de <i>Bridging</i> no VMware	22
4.3	Automação com Ansible	23
4.3.1	Estrutura do Controlo e do <i>Managed Node</i>	23
4.3.2	<i>Playbook</i> de Instalação do Docker	24
4.4	Implementação dos Serviços via Docker Compose	25
4.4.1	Arquitetura de Rede dos <i>Containers</i>	25
4.4.2	Servidor DNS: Pi-hole	26
4.4.3	Servidor Web e <i>Reverse Proxy</i> : Nginx	27
4.4.4	VPN: WireGuard	28
4.5	Implementação de CI/CD com GitLab	31
4.5.1	Configuração do GitLab Runner	31
4.5.2	Configuração da <i>Pipeline</i> : <code>.gitlab-ci.yml</code>	31
4.6	Observabilidade: Prometheus, Node Exporter e Grafana	33
4.6.1	Configuração do Prometheus	33
4.6.2	Visualização com Grafana	34
4.7	Síntese das Configurações de Segurança Aplicadas	34
5	Testes e Avaliação	37
5.1	Metodologia de Testes	37
5.2	Teste 1: Servidor DNS (Pi-hole)	38
5.3	Teste 2: Servidor Web (Nginx)	38
5.4	Teste 3: VPN (WireGuard)	39

5.5	Teste 4: <i>Pipeline</i> CI/CD	40
5.6	Teste 5: Monitorização (Prometheus + Grafana)	40
5.7	Análise Crítica dos Resultados	41
5.7.1	Objetivos Cumpridos	41
5.7.2	O que Poderia ter sido Feito de Forma Diferente	41
5.7.3	Objetivos que Ficaram por Cumprir	42
6	Conclusões	43
6.1	Síntese do Trabalho Realizado	43
6.2	Aprendizagens e Contribuições	44
6.3	Comparação com Trabalhos Relacionados	44
6.4	Trabalho Futuro	45
6.5	Considerações Finais	46
A	Proposta Original do Projeto	A1
B	Outro(s) Apêndice(s)	B1

Lista de Tabelas

2.1	Comparação entre Máquinas Virtuais e <i>Containers</i>	6
2.2	Comparação de <i>Stacks</i> de Observabilidade <i>Open Source</i>	10
3.1	Serviços a Implementar e Ferramentas Seleccionadas	14
4.1	Medidas de Segurança Aplicadas na Plataforma	35

Lista de Figuras

4.1	Verificação do endereço IP e acesso SSH inicial ao Ubuntu Server 24.04 LTS.	20
4.2	Estrutura do repositório e ficheiro de inventário do Ansible (<code>inventory.ini</code>).	24
4.3	Execução bem-sucedida do <i>Playbook</i> <code>setup_docker.yml</code> — <i>Play Recap</i> com <code>failed=0</code>	25
4.4	Interface de administração do servidor DNS (Pi-hole) a correr em <i>container</i> .	27
4.5	Página Web servida pelo Nginx através de um Volume Docker em modo <i>Read-Only</i>	28
4.6	Registo bem-sucedido do GitLab Runner no servidor Ubuntu.	31
4.7	<i>Runner</i> ativo e em comunicação com o repositório GitLab (estado <i>online</i>).	32
4.8	Execução bem-sucedida da <i>pipeline</i> de CI/CD — <i>deploy</i> automático da infraestrutura a partir de um <i>commit</i>	33
4.9	Painel <i>Node Exporter Full</i> no Grafana a exibir métricas do servidor em tempo real.	34

Capítulo 1

Introdução

A administração de sistemas e redes atravessa uma transformação profunda, impulsionada pela crescente exigência de disponibilidade contínua, escalabilidade e rastreabilidade das infraestruturas tecnológicas. A necessidade de implementar e gerir serviços de forma rápida, reproduzível e resistente a falhas humanas impulsionou a adoção em larga escala de tecnologias de *containerização* e da filosofia de *Infrastructure as Code* (IaC) [1].

1.1 Contextualização e Motivação

Numa infraestrutura tecnológica de pequena ou média dimensão — contexto comum em instituições de ensino, PMEs ou laboratórios de investigação —, a gestão manual e ad-hoc de serviços críticos como DNS, *reverse proxies*, VPNs ou servidores de ficheiros revela-se insustentável a longo prazo. Cada intervenção manual representa um risco de inconsistência: um ficheiro de configuração alterado sem versionamento, uma dependência instalada fora do contexto correto, ou um serviço repostado com parâmetros diferentes dos originais são erros frequentes em ambientes não automatizados.

A motivação deste projeto nasceu precisamente desta realidade. Ao longo do processo de implementação, foi possível constatar em primeira mão como a ausência de automação agrava os tempos de recuperação após falhas: por exemplo, a simples reinicialização do *router* da rede local foi suficiente para derrubar a conectividade da máquina virtual (VM),

exigindo uma intervenção manual de diagnóstico e reconfiguração que, num ambiente automatizado e bem documentado, teria sido evitada ou resolvida em segundos. Este tipo de incidente, que será detalhado no Capítulo 4, ilustra a relevância prática dos objetivos deste projeto.

O paradigma de *containerização*, em particular através de tecnologias como o Docker, oferece uma resposta concreta a estes desafios: as aplicações e os seus serviços são encapsulados em unidades portáteis e isoladas (*containers*), com as suas dependências declaradas de forma explícita, garantindo que o ambiente de execução é consistente independentemente do *host* físico onde corre [2]. Complementarmente, a automação de configuração com ferramentas como o Ansible e a integração de *pipelines* de CI/CD permitem que qualquer alteração à infraestrutura seja testada, auditada e aplicada de forma controlada [3].

1.2 Objetivos

O objetivo central deste projeto é construir, configurar e documentar uma plataforma de serviços de rede baseada em *containers*, com orquestração leve (Docker Compose), focada em boas práticas de segurança, CI/CD e observabilidade.

De forma mais específica, os objetivos operacionais do projeto são os seguintes:

- **Implementar serviços típicos de rede em *containers*:** servidor DNS (Pi-hole), *Reverse Proxy* HTTPS (Nginx), servidor Web estático (Nginx), VPN (WireGuard) e *stack* de monitorização (Prometheus, Node Exporter, Grafana).
- **Garantir reprodutibilidade total da infraestrutura:** toda a configuração — desde a instalação do Docker até ao *deploy* dos serviços — deve poder ser recriada a partir do repositório Git sem qualquer intervenção manual adicional, seguindo o princípio de *Single Source of Truth* [1].
- **Aplicar boas práticas de segurança:** volumes em modo *read-only*, isolamento de serviços em rede interna, gestão de segredos por variáveis de ambiente, e encriptação

de tráfego (TLS/WireGuard).

- **Automatizar o *deploy* com Ansible e CI/CD:** qualquer *commit* na *branch* principal do repositório GitLab deve despoletar automaticamente o *deploy* da infraestrutura, sem intervenção humana.
- **Integrar observabilidade centralizada:** recolha contínua de métricas de *hardware* (CPU, RAM, I/O, rede) e visualização em painéis Grafana, permitindo detetar anomalias de forma proativa.
- **Documentar o processo de *troubleshooting*:** registar e analisar os erros reais encontrados durante a implementação, demonstrando o raciocínio técnico aplicado na sua resolução.

1.3 Metodologia

A abordagem metodológica seguida neste projeto assenta em três pilares complementares. Em primeiro lugar, adotou-se uma estratégia incremental: os serviços foram implementados por camadas, começando pela base (sistema operativo e Docker), passando pelos serviços de rede (DNS, Web, VPN) e terminando na observabilidade e automação CI/CD. Esta ordem garante que cada camada superior pôde ser validada antes de avançar.

Em segundo lugar, toda a configuração foi versionada em Git desde o início, o que permitiu rastrear cada alteração e reverter configurações problemáticas de forma controlada. Por fim, os erros e os processos de *troubleshooting* foram documentados no momento em que ocorreram, preservando o contexto técnico necessário para a sua análise posterior.

1.4 Estrutura do Documento

O presente relatório está organizado da seguinte forma:

- **Capítulo 2 — Estado da Arte:** revisão das tecnologias envolvidas, com base em documentação oficial e literatura técnica, contextualizando as opções tomadas.

- **Capítulo 3 — Levantamento das Necessidades:** análise dos requisitos funcionais e não-funcionais da plataforma, e justificação das escolhas de ambiente.
- **Capítulo 4 — Arquitetura e Implementação:** descrição detalhada do processo de implementação, incluindo configurações, listagens de código e documentação dos problemas encontrados.
- **Capítulo 5 — Testes e Avaliação:** validação dos serviços implementados e discussão dos resultados.
- **Capítulo 6 — Conclusões:** síntese do trabalho realizado, análise crítica e propostas de trabalho futuro.

Capítulo 2

Estado da Arte

Este capítulo apresenta uma revisão das principais tecnologias utilizadas neste projeto, com base na documentação oficial das ferramentas, em especificações técnicas publicadas e em literatura académica da área de administração de sistemas e DevOps. O objetivo não é apenas listar ferramentas, mas compreender o contexto em que surgiram e as razões pelas quais representam o estado da arte atual para o problema em causa.

2.1 Virtualização e *Containers*: Evolução e Diferenças

A virtualização de servidores, popularizada na primeira década do século XXI com *hypervisors* como o VMware ESXi, o Hyper-V ou o KVM, revolucionou a forma como os recursos de *hardware* são partilhados. Ao permitir executar múltiplos sistemas operativos isolados numa mesma máquina física, tornou possível consolidar infraestruturas inteiras em poucos servidores físicos [4].

No entanto, as máquinas virtuais tradicionais trazem consigo um custo significativo: cada VM replica um sistema operativo completo, o que implica consumo de RAM, armazenamento e tempo de arranque na ordem dos minutos. Para ambientes que exigem escalabilidade rápida e alta densidade de serviços, este modelo revelou-se pouco eficiente.

A tecnologia de *containers* surge como resposta a esta limitação. Em vez de virtualizar o *hardware*, os *containers* virtualizam o sistema operativo: partilham o mesmo *kernel* do *host*, mas isolam os processos, o sistema de ficheiros e a rede de cada aplicação através de mecanismos do *kernel* Linux, nomeadamente os *namespaces* e os *cgroups* [2]. O resultado é um arranque em milissegundos, um consumo de recursos mínimo e uma portabilidade total entre ambientes.

A Tabela 2.1 sintetiza as principais diferenças entre os dois paradigmas.

Tabela 2.1: Comparação entre Máquinas Virtuais e *Containers*

Característica	Máquina Virtual (VM)	<i>Container</i>
Isolamento	<i>Hardware</i> virtualizado completo	<i>Namespaces</i> e <i>cgroups</i> do <i>kernel</i>
Sistema Operativo	SO convidado independente por VM	SO do <i>host</i> partilhado
Tempo de arranque	Minutos	Segundos a milissegundos
Consumo de RAM	Centenas de MB a vários GB por VM	Dezenas de MB por <i>container</i>
Portabilidade	Imagem pesada (GBs), acoplada ao <i>hypervisor</i>	Imagem leve, executável em qualquer <i>host</i> Docker
Segurança de isolamento	Elevada (separação de <i>kernel</i>)	Moderada (partilha de <i>kernel</i>)

2.2 Docker e Docker Compose

O Docker, lançado em 2013, democratizou o uso de *containers* ao fornecer uma interface de linha de comandos acessível e um ecossistema de imagens públicas (Docker Hub) [2], [5]. A sua adoção generalizada é bem ilustrada pela vasta quantidade de recursos tutoriais disponíveis, desde guias técnicos especializados [6] a conteúdo de divulgação [7]. Um *container* Docker é instanciado a partir de uma *image*, que por sua vez é construída de forma declarativa através de um ficheiro **Dockerfile**.

Para ambientes com múltiplos serviços interdependentes — como o deste projeto,

onde o DNS, o *proxy* e a monitorização precisam de comunicar entre si —, o Docker Compose permite definir toda a topologia de *containers* num único ficheiro YAML (`docker-compose.yml`) [8], [9]. Cada serviço é descrito com a sua imagem, portas expostas, variáveis de ambiente, volumes e redes, garantindo que a infraestrutura pode ser recriada com um único comando (`docker compose up -d`).

Do ponto de vista de segurança, a documentação oficial do Docker recomenda um conjunto de boas práticas que foram aplicadas neste projeto: execução de *containers* com utilizadores sem privilégios de *root*, montagem de volumes em modo *read-only* sempre que possível, e isolamento de serviços em redes internas sem exposição desnecessária de portas ao exterior [10], [11].

2.2.1 Podman como Alternativa *Rootless*

Paralelamente ao Docker, o Podman tem emergido como uma alternativa com características de segurança superiores em determinados cenários. A sua arquitetura *daemonless* — não exige um processo *daemon* com privilégios de *root* em execução permanente — reduz a superfície de ataque do sistema [12]. O Podman suporta a execução de *containers* no contexto de um utilizador sem privilégios (*rootless containers*), o que significa que mesmo que o processo do *container* seja comprometido, o atacante não herda privilégios de administrador no *host*.

Para este projeto, optou-se pelo Docker em detrimento do Podman, principalmente pela sua maturidade na integração com o Docker Compose V2 e pela compatibilidade com as imagens utilizadas (Pi-hole, WireGuard com `cap_add`). Contudo, reconhece-se que para ambientes de produção de alta segurança, o Podman com *rootless* seria a escolha mais indicada.

2.3 Infraestrutura como Código (IaC) e Ansible

O conceito de *Infrastructure as Code* (IaC) estabelece que a configuração de toda a infraestrutura deve ser descrita em ficheiros de texto estruturado, versionados num sistema de

controle de versões, e aplicados de forma automatizada e reproduzível [1]. A vantagem fundamental desta abordagem é a eliminação da dicotomia entre o que está documentado e o que está realmente instalado: o repositório Git é, simultaneamente, a documentação e o estado real da infraestrutura.

O Ansible é uma das ferramentas de IaC mais adotadas na indústria, em particular para a gestão de configuração de sistemas Linux [13]. Contrariamente a ferramentas como o Puppet ou o Chef, o Ansible não requer a instalação de agentes nos servidores alvo: comunica exclusivamente por SSH, utilizando o protocolo que já está disponível em qualquer servidor Linux por omissão. Esta característica *agentless* simplifica enormemente a adoção em infraestruturas existentes.

A propriedade mais importante do Ansible do ponto de vista arquitetural é a **idempotência** [14], [15]: um *Playbook* pode ser executado múltiplas vezes contra o mesmo servidor e o resultado é sempre o mesmo estado final, independentemente do estado inicial. Se o Docker já estiver instalado, o Ansible não o reinstala; se o serviço já estiver ativo, não o reinicia desnecessariamente. Esta propriedade é fundamental para a integração com pipelines de CI/CD, onde os *Playbooks* são executados automaticamente a cada *commit*.

2.4 Serviços de Rede em *Containers*

2.4.1 DNS com Pi-hole

O Sistema de Nomes de Domínio (DNS), especificado no RFC 1035 [16], é o protocolo fundamental que traduz nomes de domínio legíveis por humanos em endereços IP. Numa rede interna, um servidor DNS local oferece vantagens significativas: resolução de nomes privados (como `datacenter.local`), caching de respostas para redução de latência, e controle sobre as resoluções realizadas.

O Pi-hole [17], [18], [19] é um servidor DNS de código aberto que, para além das funções de resolução e cache, implementa bloqueio de domínios a nível de rede. Ao contrário dos

bloqueadores de anúncios que funcionam ao nível do navegador, o Pi-hole bloqueia pedidos DNS antes de qualquer ligação ser estabelecida, afetando todos os dispositivos da rede. Neste projeto, o Pi-hole foi utilizado principalmente pelas suas capacidades de servidor DNS autoritativo e recursivo para a rede interna.

2.4.2 *Reverse Proxy* com Nginx

O Nginx [20], [21] é um servidor HTTP de alto desempenho, amplamente utilizado como *reverse proxy* em infraestruturas modernas. No contexto deste projeto, o Nginx atua como o único ponto de entrada da rede, recebendo todos os pedidos nas portas 80 (HTTP) e 443 (HTTPS) e encaminhando-os para os serviços internos correspondentes. Esta arquitetura de ponto de entrada único (*single ingress point*) simplifica a gestão de certificados TLS e permite centralizar as políticas de segurança.

2.4.3 VPN com WireGuard

O WireGuard é um protocolo de VPN moderno, desenvolvido por Jason A. Donenfeld e apresentado em 2017 na conferência NDSS [22]. Para além da especificação técnica original, existem recursos acessíveis que demonstram a sua configuração prática [23], [24]. Em comparação com protocolos estabelecidos como o OpenVPN ou o IPSec, o WireGuard apresenta uma base de código significativamente menor (aproximadamente 4.000 linhas de código C, contra centenas de milhares no OpenVPN), o que reduz a superfície de ataque e facilita as auditorias de segurança. O protocolo foi integrado no *kernel* Linux na versão 5.6 (2020), tornando-o parte do sistema operativo e eliminando a dependência de módulos externos.

Do ponto de vista criptográfico, o WireGuard utiliza primitivas modernas e bem auditadas: Curve25519 para o acordo de chaves, ChaCha20-Poly1305 para encriptação autenticada, e BLAKE2s para *hashing* [22]. A configuração é substancialmente mais simples do que a do OpenVPN, resumindo-se essencialmente à troca de chaves públicas entre os *peers*.

2.5 Observabilidade e Monitorização

Em arquiteturas modernas baseadas em *containers*, a monitorização tradicional de recursos (CPU/RAM) deixou de ser suficiente. O conceito de **observabilidade**, popularizado pela comunidade de *Site Reliability Engineering* (SRE) da Google [25], estabelece que um sistema é considerado observável quando é possível inferir o seu estado interno a partir dos seus dados externos, sem necessidade de intervenção direta.

Os três pilares da observabilidade são as métricas, os *logs* e os *traces* [25], [26]. Este projeto aborda os dois primeiros:

- **Métricas (Prometheus + Node Exporter):** O Prometheus [27] é uma base de dados de séries temporais desenhada especificamente para a recolha de métricas de sistemas distribuídos. O seu modelo de *pull* — o Prometheus interroga ativamente os *targets* em intervalos regulares — simplifica a arquitetura e permite detetar facilmente quando um serviço fica inacessível. O Node Exporter [28] é um agente leve que expõe métricas de *hardware* do sistema operativo (utilização de CPU, memória, I/O de disco, interfaces de rede) num formato compreensível pelo Prometheus.
- **Visualização (Grafana):** O Grafana [29] é uma plataforma de visualização de dados que se conecta ao Prometheus como fonte de dados e permite construir painéis interativos com gráficos de séries temporais, indicadores de estado e alertas.

A Tabela 2.2 apresenta uma comparação das principais *stacks* de observabilidade *open source* atualmente disponíveis.

Tabela 2.2: Comparação de *Stacks* de Observabilidade *Open Source*

Stack	Métricas	Logs	Complexidade
Prometheus + Grafana	Prometheus	Loki (opcional)	Baixa a média
ELK (Elastic Stack)	Metricbeat	Elasticsearch + Kibana	Elevada
EFK (Elastic + Fluentd)	Metricbeat	Fluentd + Elasticsearch	Elevada
Grafana LGTM	Mimir	Loki	Média

Para este projeto, a escolha da *stack* Prometheus + Grafana justifica-se pela sua baixa complexidade de configuração em ambiente *container*, pela excelente documentação oficial

[27], [29] e pela elevada adoção na comunidade, que facilita a resolução de problemas. O painel *Node Exporter Full* [30], com o ID 1860 no repositório oficial do Grafana, permite uma visualização completa das métricas de sistema sem qualquer configuração adicional.

2.6 CI/CD e GitLab

O paradigma de *Continuous Integration / Continuous Deployment* (CI/CD) estabelece que qualquer alteração ao código-fonte deve ser automaticamente testada e, se os testes passarem, aplicada ao ambiente de produção sem intervenção humana [3]. No contexto de infraestrutura como código, este princípio significa que qualquer alteração a um ficheiro de configuração ou *Playbook* Ansible, quando submetida ao repositório, desencadeia automaticamente o re-*deploy* da infraestrutura.

O GitLab [31], [32], [33] oferece um sistema de CI/CD integrado com o repositório Git, baseado num ficheiro de orquestração (`.gitlab-ci.yml`) que define os *jobs* a executar. A execução dos *jobs* é delegada a processos designados *Runners*, que podem ser geridos pela própria plataforma GitLab (SaaS) ou instalados localmente (*self-hosted*). Para este projeto, optou-se por um *Runner* local instalado no próprio servidor Ubuntu, uma vez que a infraestrutura se encontra numa rede privada sem acesso direto a *runners* externos.

Capítulo 3

Levantamento das Necessidades Funcionais

Com base no guião do projeto proposto, o objetivo central consiste na criação de um *data center* de serviços de rede que possa ser gerido de forma reprodutível, recorrendo exclusivamente a ferramentas *open source*. As necessidades do projeto dividem-se em duas categorias principais: os serviços a disponibilizar (requisitos funcionais) e os requisitos técnicos e de arquitetura (requisitos não-funcionais).

3.1 Serviços de Rede a Disponibilizar

A infraestrutura deve contemplar um conjunto de serviços típicos de uma rede de pequena/média dimensão. A Tabela 3.1 apresenta os serviços a implementar, a ferramenta escolhida para cada um e a justificação técnica da escolha.

De forma mais detalhada, os requisitos funcionais de cada serviço são os seguintes:

- **Servidor DNS:** Resolução de nomes para os hosts e serviços da rede interna; funcionamento em modo recursivo (reencaminhamento para servidores *upstream* públicos para domínios externos); interface de administração acessível por HTTP.
- **Reverse Proxy HTTPS:** Ponto de entrada único na rede; encaminhamento de

Tabela 3.1: Serviços a Implementar e Ferramentas Seleccionadas

Serviço	Ferramenta	Justificação
Servidor DNS	Pi-hole	Servidor recursivo e autoritativo; interface de gestão web integrada; bloqueio de domínios a nível de rede [17]
<i>Reverse Proxy</i> HTTPS	Nginx	Elevado desempenho; suporte nativo a TLS; configuração declarativa simples [20]
Servidor Web estático	Nginx	Reutilização da mesma imagem do <i>proxy</i> ; servir conteúdo estático com mínimo overhead
VPN	WireGuard	Protocolo moderno com código reduzido (menor superfície de ataque); integrado no <i>kernel</i> Linux; configuração simples [22]
Recolha de métricas	Prometheus + Node Exporter	Stack open source com elevada adoção; modelo pull; baixa complexidade de configuração [27]
Visualização	Grafana	Compatibilidade nativa com Prometheus; painéis pré-construídos para Node Exporter [29]

pedidos HTTP/HTTPS para os serviços internos corretos com base no nome de domínio ou no caminho do pedido; terminação TLS centralizada.

- **Servidor Web:** Disponibilização de uma página de estado da infraestrutura, servida através do Nginx com conteúdo injetado por volume Docker.
- **VPN (WireGuard):** Acesso remoto seguro à rede interna de *containers* a partir de dispositivos externos (computador portátil e telemóvel); criação automática de perfis de cliente (QR codes para configuração no telemóvel).
- **Monitorização:** Recolha contínua de métricas de *hardware* do servidor *host* (CPU, RAM, I/O de disco, interfaces de rede); armazenamento em base de dados de séries temporais; visualização em painéis interativos.

3.2 Requisitos Técnicos e de Arquitetura

Para além do correto funcionamento dos serviços, a plataforma tem de respeitar rigorosos critérios de implementação e gestão, tal como estabelecido no guião do projeto.

- **Containerização e Configuração Declarativa:** Todos os serviços têm de ser executados em *containers* Docker. A configuração deve ser feita de forma declarativa (ficheiro `docker-compose.yml`), garantindo a reprodutibilidade e o versionamento [8].
- **Observabilidade e Monitorização:** É obrigatória a integração de uma *stack open source* para a recolha contínua de métricas e configuração de alertas básicos (latência, erros 5xx, saturação de CPU/RAM).
- **Segurança (*Hardenig*):** O projeto exige a aplicação do princípio do mínimo privilégio nos *containers*, gestão segura e isolada de segredos, encriptação obrigatória de tráfego (TLS/WireGuard) e uma política de atualização automatizada das imagens dos *containers* [11].
- **Pipeline de CI/CD:** Deve ser implementada uma *pipeline* simples (utilizando o Git e um *runner* local) para testar automaticamente as alterações de configuração e assegurar um *deploy* controlado para o ambiente de laboratório [3].

3.3 Ambiente Base: Virtualização e Sistema Operativo

Para suportar a plataforma e garantir o isolamento do *host* físico, a infraestrutura base foi implementada em ambiente virtualizado.

- **Virtualização (VMware Workstation):** A plataforma escolhida para o *hypervisor* local é o VMware Workstation [4]. Esta escolha justifica-se pela sua estabilidade,

pela excelente gestão de redes virtuais (modos *Bridged* e NAT) necessárias para simular tráfego externo/interno, e pela eficiência na alocação de recursos. O modo *Bridged* foi o selecionado para este projeto, pois permite que a VM obtenha um endereço IP diretamente na rede física local, sendo acessível por todos os dispositivos da rede como se fosse uma máquina física real — comportamento essencial para o funcionamento correto do DNS e da VPN.

- **Sistema Operativo (*Guest OS*):** Para o sistema operativo que aloja os *containers*, optou-se pelo **Ubuntu Server 24.04 LTS** [34]. Esta distribuição foi selecionada por possuir suporte de longo prazo (até 2029), excelente compatibilidade nativa com Docker e Ansible, e por não incluir interface gráfica (*headless*), minimizando a superfície de ataque e o consumo de recursos da VM.

3.3.1 Configuração de Rede do Servidor

A configuração de rede do Ubuntu Server 24.04 é gerida pelo **Netplan** [35], um sistema de configuração de rede declarativo que gera configurações para o *backend networkd* ou **NetworkManager**. Para garantir que o servidor mantém sempre o mesmo endereço IP — condição essencial para o correto funcionamento do DNS e da VPN —, optou-se por configurar um IP estático (192.168.1.252) em vez de depender de DHCP.

A configuração da rede é definida em `/etc/netplan/00-installer-config.yaml` e, após validação com `netplan try`, aplicada com `netplan apply`. O Capítulo 4 detalha os erros encontrados durante este processo e as soluções aplicadas.

3.3.2 Acesso Externo: Router em modo DMZ

Para permitir o acesso externo à VPN WireGuard a partir de redes externas, foi necessário configurar o *router* da rede doméstica (MEO) em modo DMZ (*Demilitarized Zone*) apontado para o IP estático do servidor (192.168.1.252). Esta configuração faz com que todo o tráfego externo não solicitado seja reencaminhado para o servidor, permitindo que o WireGuard (porta UDP 51820) receba ligações de clientes externos.

O IP público do servidor, necessário para a configuração do WireGuard (`SERVERURL`), foi obtido através do serviço `ip.me` e registado no ficheiro `docker-compose.yml`.

Capítulo 4

Arquitetura e Implementação

Este capítulo detalha o processo prático de implementação da plataforma de serviços de rede, descrevendo não apenas as configurações finais, mas também os problemas reais encontrados e o raciocínio técnico aplicado na sua resolução. A abordagem adotada seguiu as melhores práticas de *Infrastructure as Code* (IaC), garantindo que todo o ambiente pudesse ser reconstruído de forma automatizada e previsível [1].

4.1 Arquitetura Geral da Plataforma

Antes de detalhar a implementação, importa descrever a arquitetura global da plataforma. Esta organiza-se em três camadas hierárquicas com responsabilidades bem definidas:

1. **Camada de infraestrutura:** a máquina física Windows com VMware Workstation a correr uma VM Ubuntu Server 24.04 LTS em modo *Bridged* na rede local.
2. **Camada de serviços:** os *containers* Docker (Nginx, Pi-hole, WireGuard, Prometheus, Node Exporter, Grafana) a comunicar através de uma rede interna Docker (`rede_interna`).
3. **Camada de automação:** o Ansible (executado via WSL na máquina Windows) e a *pipeline* GitLab CI/CD (executada pelo Runner instalado no servidor Ubuntu).

Todo o tráfego externo entra pelo Nginx nas portas 80/443, ou diretamente pelo WireGuard na porta UDP 51821. Os serviços internos comunicam exclusivamente através da rede Docker `bridge` isolada, sem exposição desnecessária de portas ao exterior.

4.2 Configuração do Ambiente Base (Ubuntu Server)

4.2.1 Instalação e Configuração Inicial

O Ubuntu Server 24.04 LTS foi instalado numa VM VMware com as seguintes especificações: 2 vCPUs, 4 GB de RAM e 50 GB de disco. A interface de rede foi configurada em modo *Bridged* no VMware, o que permite à VM participar diretamente na rede local e obter um endereço IP nessa gama.

Após a instalação, o primeiro passo foi verificar a conectividade de rede e o endereço IP atribuído, conforme ilustrado na Figura 4.1.

```
root@ubuntu-server:~# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:b4:12:59 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.1.252/24 metric 100 brd 192.168.1.255 scope global dynamic ens33
        valid_lft 3537sec preferred_lft 3537sec
    inet6 2001:8a0:fc04:9200:20c:29ff:feb4:1259/64 scope global dynamic mngtmpaddr noprefixroute
        valid_lft 89938sec preferred_lft 89938sec
    inet6 fe80::20c:29ff:feb4:1259/64 scope link
        valid_lft forever preferred_lft forever
root@ubuntu-server:~#
```

Figura 4.1: Verificação do endereço IP e acesso SSH inicial ao Ubuntu Server 24.04 LTS.

4.2.2 Configuração de IP Estático com Netplan

Para garantir que o servidor mantém sempre o mesmo endereço IP na rede local — condição essencial para o correto funcionamento do DNS, da VPN e das entradas do inventário Ansible —, procedeu-se à configuração de um IP estático através do Netplan [35].

Problema Encontrado: Erros de Indentação YAML e Falhas de DNS

A configuração do Netplan revelou-se o primeiro obstáculo técnico significativo do projeto. O ficheiro de configuração `/etc/netplan/00-installer-config.yaml` utiliza o formato YAML, que é extremamente sensível à indentação: um espaço extra ou a substituição de espaços por tabulações resulta num erro de *parse* que impede a aplicação da configuração.

Durante a configuração, foram cometidos dois erros distintos:

1. **Erro de indentação:** A secção `nameservers` foi indentada ao nível errado (ao nível de `ethernets` em vez de ao nível da interface específica), resultando no erro:

```
Invalid YAML at /etc/netplan/00-installer-config.yaml
```

O problema foi diagnosticado com o comando `netplan try`, que aplica a configuração temporariamente durante 120 segundos e reverte automaticamente se não for confirmada — mecanismo de segurança crucial para evitar perda de acesso remoto ao servidor.

2. **Falha de resolução DNS:** Após corrigir a indentação e aplicar o IP estático, o servidor perdeu capacidade de resolução de nomes de domínio. A causa foi a omissão dos servidores `nameservers` na configuração Netplan. O servidor tentava resolver nomes através de si próprio (o Pi-hole ainda não estava instalado), resultando em falhas de DNS. A solução imediata foi adicionar explicitamente os servidores DNS do Google (8.8.8.8 e 8.8.4.4) na configuração Netplan, até o Pi-hole estar operacional.

A configuração Netplan final, corretamente indentada, ficou com o seguinte aspeto:

Listing 4.1: Configuração Netplan para IP estático

```
network:
  version: 2
  ethernet:
    ens33:
```

```
dhcp4: no
addresses:
  - 192.168.1.252/24
routes:
  - to: default
    via: 192.168.1.1
nameservers:
  addresses:
    - 8.8.8.8
    - 8.8.4.4
```

4.2.3 Problema de *Bridging* no VMware

Um segundo problema de rede surgiu de forma intermitente após reinícios do *router* da rede doméstica. A VM perdia subitamente a conectividade com a rede física, apesar do Netplan estar corretamente configurado.

Diagnóstico

O diagnóstico começou por verificar o estado da interface de rede no servidor (`ip addr show ens33` e `ip route show`), que apresentava a configuração correta. O problema residia, portanto, na camada de virtualização, e não no sistema operativo convidado.

Ao inspecionar o **VMware Virtual Network Editor** (ferramenta de gestão de redes virtuais do VMware, acessível em *Edit > Virtual Network Editor*), identificou-se a causa: o VMware Workstation mantém uma lista de adaptadores de rede físicos disponíveis para o modo *Bridged*, e após um reinício do *router*, a ferramenta tinha reordenado os adaptadores e selecionado automaticamente um adaptador virtual do Windows (criado pelo **Hyper-V** ou pelo cliente **OpenVPN**) em vez da placa de rede física Realtek.

Esta seleção incorreta significa que a VM tentava fazer a ponte de rede (*bridge*) para um adaptador virtual sem ligação física real à rede, tornando-a invisível para os restantes

dispositivos da LAN.

Solução

A solução consistiu em forçar manualmente o VMware a utilizar o adaptador correto no Virtual Network Editor:

1. Abrir o VMware Virtual Network Editor com privilégios de administrador (*Change Settings*).
2. Selecionar a rede **VMnet0** (utilizada pelo modo *Bridged*).
3. Em vez de *Automatic*, selecionar explicitamente o adaptador físico **Realtek** (o adaptador Ethernet físico da máquina).
4. Aplicar as alterações e reiniciar a VM.

Após esta correção, a VM passou a manter a conectividade de rede de forma estável, independentemente de reinícios do *router*.

4.3 Automação com Ansible

4.3.1 Estrutura do Controlo e do *Managed Node*

Para cumprir o requisito de reprodutibilidade, todas as configurações e instalações no servidor foram realizadas exclusivamente através do Ansible, sem qualquer intervenção manual direta. O Ansible foi instalado no ambiente **WSL (Windows Subsystem for Linux)** [36] na máquina de administração Windows, dado que o Ansible não tem suporte nativo para Windows como *control node*.

O WSL fornece um ambiente Linux completo (Ubuntu 22.04) integrado no Windows, permitindo executar o Ansible e estabelecer ligações SSH ao servidor Ubuntu de forma transparente. A Figura 4.2 ilustra a estrutura do repositório e o ficheiro de inventário.

O ficheiro `inventory.ini` define o servidor de destino:

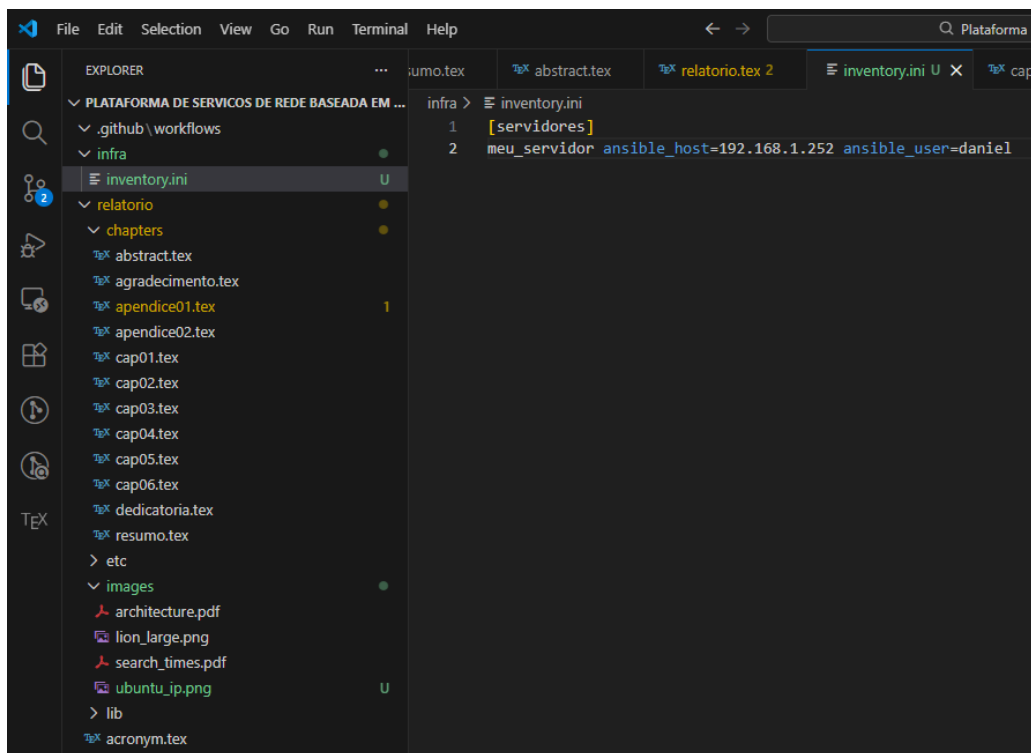


Figura 4.2: Estrutura do repositório e ficheiro de inventário do Ansible (`inventory.ini`).

Listing 4.2: Ficheiro `inventory.ini`

```
[servidores]
meu_servidor ansible_host=192.168.1.252 ansible_user=daniel
```

4.3.2 *Playbook* de Instalação do Docker

O *Playbook* `setup_docker.yml` é responsável por preparar o servidor para receber os *containers*. As suas tarefas são executadas com `become: yes` (equivalente a `sudo`), garantindo os privilégios necessários para instalação de pacotes de sistema [13].

As tarefas do *Playbook* são executadas pela seguinte ordem:

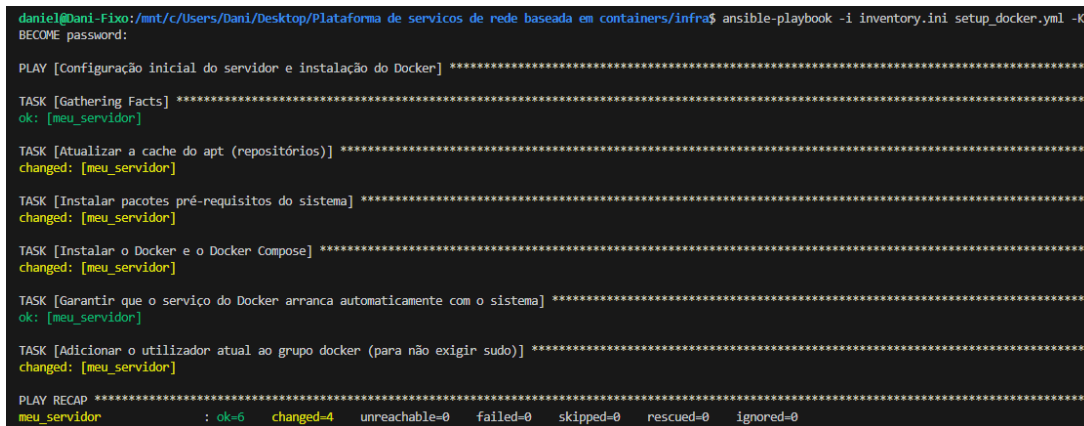
1. Atualização da *cache* APT (`apt: update_cache: yes`).
2. Instalação de dependências de sistema (`apt: apt=transport-https, ca-certificates, curl, software-properties-common`).

3. Instalação do Docker Engine (`docker.io`) e do *plugin* Docker Compose V2 (`docker-compose-v2`).
4. Ativação e início do *daemon* Docker via `systemd`, com arranque automático no boot.
5. Adição do utilizador `daniel` ao grupo `docker`, implementando o princípio do mínimo privilégio: elimina a necessidade de usar `sudo` a cada comando Docker.

A execução é iniciada com:

```
ansible-playbook -i inventory.ini setup_docker.yml
```

A Figura 4.3 ilustra o *Play Recap* de uma execução bem-sucedida, demonstrando a idempotência da ferramenta: `changed=0` indica que o Docker já se encontrava instalado e nenhuma alteração foi necessária.



```
daniel@Dani-Fixo:/mnt/c/Users/Dani/Desktop/Plataforma de servicos de rede baseada em containers/infra$ ansible-playbook -i inventory.ini setup_docker.yml -K
BECOME password:

PLAY [Configuração inicial do servidor e instalação do Docker] *****

TASK [Gathering Facts] *****
ok: [meu_servidor]

TASK [Atualizar a cache do apt (repositórios)] *****
changed: [meu_servidor]

TASK [Instalar pacotes pré-requisitos do sistema] *****
changed: [meu_servidor]

TASK [Instalar o Docker e o Docker Compose] *****
changed: [meu_servidor]

TASK [Garantir que o serviço do Docker arranqa automaticamente com o sistema] *****
ok: [meu_servidor]

TASK [Adicionar o utilizador atual ao grupo docker (para não exigir sudo)] *****
changed: [meu_servidor]

PLAY RECAP *****
meu_servidor      : ok=6  changed=4  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0
```

Figura 4.3: Execução bem-sucedida do *Playbook* `setup_docker.yml` — *Play Recap* com `failed=0`.

4.4 Implementação dos Serviços via Docker Compose

4.4.1 Arquitectura de Rede dos *Containers*

Todos os serviços são definidos no ficheiro `docker-compose.yml` e partilham uma rede interna Docker designada `rede_interna`, do tipo `bridge`. Esta rede isola os *containers* da

rede do *host* e permite-lhes comunicar entre si através de nomes de serviço (DNS interno do Docker), sem exposição desnecessária de portas ao exterior [10].

Apenas as portas estritamente necessárias são publicadas no *host*: 80 e 443 (Nginx), 53 TCP/UDP (Pi-hole), 8080 (painel Pi-hole), 9090 (Prometheus), 9100 (Node Exporter), 3000 (Grafana) e 51821 UDP (WireGuard).

4.4.2 Servidor DNS: Pi-hole

O Pi-hole é configurado como *container* na rede `rede_interna`, expondo as portas 53 TCP e UDP para resolução DNS, e a porta 8080 para o painel de administração web [17].

Problema: Palavra-passe não Aplicada pelo Volume Persistente

Durante a primeira inicialização do Pi-hole, verificou-se que a variável de ambiente `WEBPASSWORD` definida no `docker-compose.yml` não estava a ser aplicada: o painel de administração recusava a autenticação com a palavra-passe configurada.

A causa foi identificada após análise dos *logs* do *container* (`docker logs dns_pihole`): quando o volume do Pi-hole já existe com configuração persistida de uma inicialização anterior (mesmo que com configurações padrão), a variável `WEBPASSWORD` é ignorada pelo *entrypoint* da imagem.

A resolução passou por injetar o comando de definição de palavra-passe diretamente no *container* em execução, utilizando `docker exec`:

Listing 4.3: Comando de troubleshooting para definir password no Pi-hole

```
sudo docker exec dns_pihole pihole setpassword 'nova_password'
```

Este tipo de intervenção pontual demonstra a flexibilidade das arquiteturas baseadas em *containers*: é possível intervir cirurgicamente dentro do ecossistema do serviço sem comprometer o estado do *host* ou dos restantes serviços. Para evitar recorrência deste problema, a solução definitiva seria eliminar o volume antes de uma nova inicialização limpa: `docker compose down -v`.

A Figura 4.4 comprova o acesso bem-sucedido ao painel de administração do Pi-hole.



Figura 4.4: Interface de administração do servidor DNS (Pi-hole) a correr em *container*.

4.4.3 Servidor Web e *Reverse Proxy*: Nginx

O Nginx é configurado com um volume Docker em modo *read-only* (:ro) que mapeia a pasta local `./web` para o diretório de conteúdo estático do Nginx (`/usr/share/nginx/html`) [20]. Esta abordagem tem dois benefícios:

1. **Desacoplamento de conteúdo e infraestrutura:** o conteúdo HTML pode ser atualizado sem necessidade de reconstruir ou reiniciar o *container*.
2. **Segurança por *read-only*:** em conformidade com o princípio do mínimo privilégio [11], o *container* Nginx não tem permissões de escrita sobre os ficheiros que serve. Num cenário hipotético de comprometimento do processo Nginx, um atacante não conseguiria modificar os ficheiros de conteúdo.

A Figura 4.5 demonstra a página de estado da infraestrutura servida pelo Nginx.



Figura 4.5: Página Web servida pelo Nginx através de um Volume Docker em modo *Read-Only*.

4.4.4 VPN: WireGuard

O WireGuard foi implementado utilizando a imagem `lscr.io/linuxserver/wireguard`, mantida pela comunidade LinuxServer.io [24], que simplifica a geração automática de configurações de cliente relativamente à imagem oficial [22]. As configurações mais relevantes do serviço são:

- `SERVERURL=2.83.165.235`: IP público do servidor, necessário para que os clientes externos saibam para onde se ligar.
- `SERVERPORT=51821`: porta UDP utilizada pelo WireGuard nesta instalação. A porta padrão (51820) foi alterada para 51821 após se detetar um conflito com outro serviço no *router* MEO, que filtrava especificamente essa porta — este ajuste foi registado no repositório com a mensagem de *commit fix*: `muda porta da VPN para 51821 devido a conflito no router` (30 de abril de 2026).

- **PEERS=3:** gera automaticamente três perfis de cliente, incluindo ficheiros de configuração e QR codes. O número de *peers* foi aumentado de 2 para 3 numa iteração posterior (*commit feat: atualiza IP publico e aumenta peers da VPN*), para acomodar um dispositivo adicional.
- **PEERDNS=192.168.1.252:** obriga os clientes VPN a utilizar o Pi-hole como servidor DNS quando ligados, garantindo que o bloqueio de domínios e a resolução de nomes internos funcionam também em modo VPN.
- **ALLOWEDIPS=0.0.0.0/0:** todo o tráfego do cliente é encaminhado pelo túnel VPN (*full tunnel*), garantindo que nenhum tráfego escapa sem encriptação.

O *container* WireGuard requer duas *capabilities* adicionais do sistema operativo (**cap_add: NET_ADMIN, SYS_MODULE**), necessárias para manipular as interfaces de rede do *kernel* e carregar o módulo WireGuard.

Problema: Clientes Externos Não Conseguiram Ligar à VPN

Após configurar o WireGuard e gerar os perfis de cliente, o telemóvel não conseguia estabelecer a ligação VPN a partir de uma rede externa (rede 4G). O indicador de estado da aplicação WireGuard no telemóvel mostrava o estado *handshaking* de forma persistente, sem completar a ligação.

Processo de Diagnóstico

O processo de diagnóstico foi estruturado em camadas, do servidor para o exterior:

1. Verificar se o *container* WireGuard está em execução e à escuta:

```
docker ps | grep wireguard
ss -ulnp | grep 51821
```

O resultado confirmou que o *container* estava ativo e a porta 51821 UDP estava à escuta.

2. Verificar se os pacotes chegam ao servidor com tcpdump:

A ferramenta `tcpdump` permite capturar e analisar o tráfego de rede em tempo real, sendo essencial para distinguir se o problema reside no servidor ou na rede/*router* [37]. Executou-se o seguinte comando no servidor, enquanto o telemóvel tentava ligar:

Listing 4.4: Captura de tráfego WireGuard com `tcpdump`

```
sudo tcpdump -i ens33 udp port 51821 -n
```

O resultado foi revelador: **zero pacotes capturados**. Isto significava que os pacotes enviados pelo telemóvel não estavam a chegar à interface de rede do servidor, eliminando qualquer hipótese de problema na configuração do WireGuard ou do *container*.

3. Identificação do problema: router a bloquear tráfego externo:

A ausência de pacotes na interface `ens33` indicava que o *router* MEO estava a filtrar o tráfego UDP externo antes de o encaminhar para a rede interna. A solução foi configurar o *router* em modo **DMZ** apontado para o IP estático do servidor (`192.168.1.252`), fazendo com que todo o tráfego externo não solicitado seja diretamente reencaminhado para o servidor.

Solução aplicada:

1. Aceder à interface de administração do *router* MEO (`192.168.1.1`).
2. Ativar o modo DMZ e configurar o IP de destino: `192.168.1.252`.
3. Reiniciar fisicamente o *router* (um reinício por software não foi suficiente para aplicar a configuração DMZ).

Após o reinício físico do *router*, repetiu-se o `tcpdump`: os pacotes UDP do telemóvel passaram a ser visíveis na interface `ens33`, e a ligação VPN foi estabelecida com sucesso.

4.5 Implementação de CI/CD com GitLab

4.5.1 Configuração do GitLab Runner

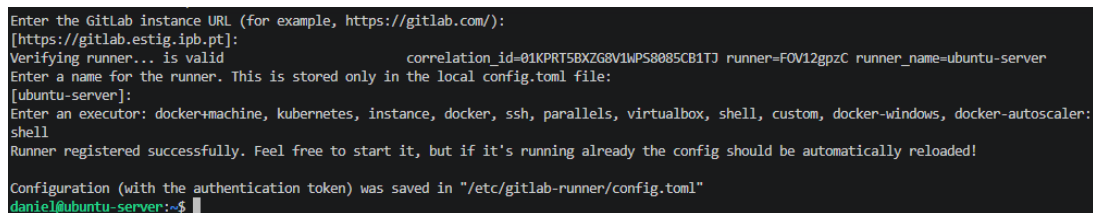
Para implementar a *pipeline* de CI/CD, foi necessário instalar um *GitLab Runner* no próprio servidor Ubuntu [31]. A opção por um *runner* local justifica-se pela arquitetura de rede: o servidor encontra-se numa rede privada, inacessível diretamente pelos *runners* partilhados da plataforma GitLab. O modelo *Pull* — onde o *Runner* interpela periodicamente o servidor GitLab para obter novos *jobs* — resolve este problema sem expor o servidor à Internet.

O *Runner* foi instalado e registado com o executor `shell`:

Listing 4.5: Registo do GitLab Runner

```
sudo gitlab-runner register \
  --url https://gitlab.estig.ipb.pt/ \
  --registration-token <TOKEN> \
  --executor shell \
  --description "runner-datacenter"
```

A Figura 4.6 ilustra o processo de registo bem-sucedido.



```
Enter the GitLab instance URL (for example, https://gitlab.com/):
[https://gitlab.estig.ipb.pt]:
Verifying runner... is valid correlation_id=01KPRT5BXZG8V1WP58085CB1TJ runner=FOV12gpzC runner_name=ubuntu-server
Enter a name for the runner. This is stored only in the local config.toml file:
[ubuntu-server]:
Enter an executor: docker+machine, kubernetes, instance, docker, ssh, parallels, virtualbox, shell, custom, docker-windows, docker-autoscaler:
shell
Runner registered successfully. Feel free to start it, but if it's running already the config should be automatically reloaded!
Configuration (with the authentication token) was saved in "/etc/gitlab-runner/config.toml"
daniel@ubuntu-server:~$
```

Figura 4.6: Registo bem-sucedido do GitLab Runner no servidor Ubuntu.

4.5.2 Configuração da *Pipeline*: `.gitlab-ci.yml`

O ficheiro `.gitlab-ci.yml` define a *pipeline* de CI/CD. A sua lógica é simples e deliberada: a cada *commit* na *branch* principal, o *Runner* executa o *Playbook* Ansible

Runners

Runners are processes that pick up and execute CI/CD jobs for GitLab. [What is GitLab Runner?](#)

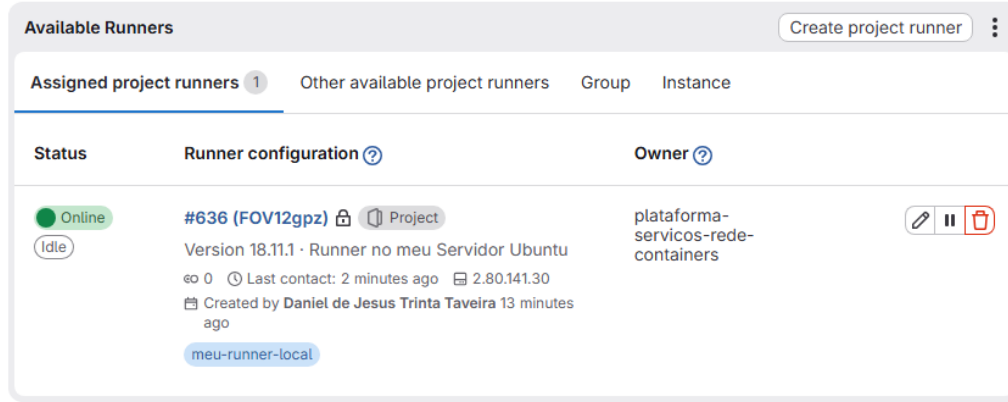


Figura 4.7: *Runner* ativo e em comunicação com o repositório GitLab (estado *online*).

`deploy_servicos.yml` localmente no servidor (usando `-i "localhost,-c local`), garantindo que os *containers* são atualizados sem intervenção humana.

Listing 4.6: Ficheiro `.gitlab-ci.yml`

```
stages:
  - deploy

deploy_infraestrutura:
  stage: deploy
  tags:
    - meu-runner-local
  script:
    - echo "A iniciar o deploy automatico via CI/CD..."
    - cd infra
    - sudo ansible-playbook -i "localhost," -c local
      deploy_servicos.yml
    - echo "Deploy concluido com sucesso!"
```

Para que o utilizador do *Runner* possa executar o Ansible com `sudo` sem intervenção

manual, foi necessário adicionar uma regra no **sudoers**:

```
gitlab-runner ALL=(ALL) NOPASSWD: /usr/bin/ansible-playbook
```

A Figura 4.8 demonstra a execução completa e bem-sucedida da *pipeline*.

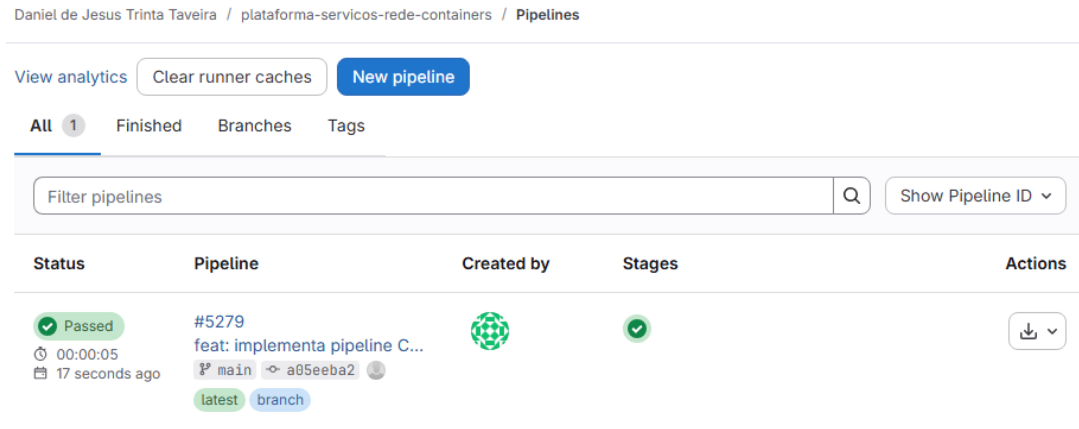


Figura 4.8: Execução bem-sucedida da *pipeline* de CI/CD — *deploy* automático da infraestrutura a partir de um *commit*.

4.6 Observabilidade: Prometheus, Node Exporter e Grafana

4.6.1 Configuração do Prometheus

O Prometheus é configurado através do ficheiro `prometheus.yml`, montado como volume *read-only* no *container* [27]. O ficheiro define dois *scrape jobs*:

- **prometheus**: recolhe métricas do próprio Prometheus (porto 9090), permitindo monitorizar o estado do sistema de monitorização.
- **node-exporter**: recolhe métricas de *hardware* do servidor através do Node Exporter (porto 9100). O *target* é referenciado pelo nome do serviço Docker (`node-exporter:9100`), utilizando o DNS interno da rede Docker.

O intervalo de *scrape* foi configurado a 15 segundos, proporcionando granularidade suficiente para detetar anomalias sem sobrecarregar a base de dados.

4.6.2 Visualização com Grafana

O Grafana foi configurado com a palavra-passe de administrador inicial definida por variável de ambiente (`GF_SECURITY_ADMIN_PASSWORD`) [29]. Após o primeiro acesso, foi adicionado o Prometheus como fonte de dados (*Data Source*) e importado o painel pré-construído **Node Exporter Full** (ID 1860 no repositório oficial do Grafana), que exhibe de forma visual todas as métricas de sistema.

A Figura 4.9 ilustra o painel de monitorização em tempo real.

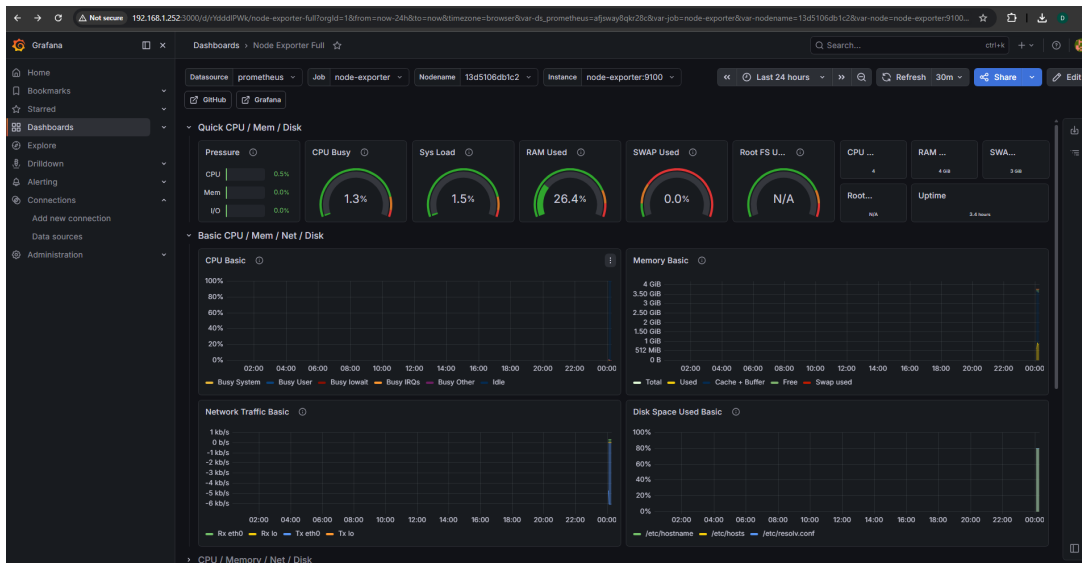


Figura 4.9: Painel *Node Exporter Full* no Grafana a exibir métricas do servidor em tempo real.

4.7 Síntese das Configurações de Segurança Aplicadas

A Tabela 4.1 sintetiza as medidas de segurança aplicadas na plataforma, em conformidade com as recomendações do NIST SP 800-190 [11].

Tabela 4.1: Medidas de Segurança Aplicadas na Plataforma

Princípio	Implementação	Serviço(s) afetado(s)
Mínimo privilégio	Volumes montados em modo <i>read-only</i> (:ro)	Nginx (Web), Prometheus
Mínimo privilégio	Utilizador daniel no grupo docker (sem sudo para Docker)	Todos
Isolamento de rede	Rede Docker interna (bridge) sem exposição desnecessária	Todos
Encriptação de tráfego	WireGuard (ChaCha20-Poly1305) para acesso remoto	WireGuard
Encriptação de tráfego	HTTPS (TLS) via Nginx (a implementar com Let's Encrypt)	Nginx
Isolamento de <i>capabilities</i>	cap_add limitado ao estritamente necessário	WireGuard
Gestão de segredos	Palavras-passe por variáveis de ambiente (não em código)	Pi-hole, Grafana

Capítulo 5

Testes e Avaliação

Este capítulo apresenta os testes realizados para verificar que a plataforma implementada cumpre os objetivos definidos no Capítulo 3. Para cada serviço, descreve-se o método de teste, o resultado esperado e o resultado obtido. Complementarmente, discutem-se os resultados de forma crítica, identificando o que funcionou conforme esperado, o que exigiu ajustes e o que permanece como trabalho futuro.

5.1 Metodologia de Testes

Os testes foram estruturados em dois níveis:

- **Testes de funcionamento individual (*smoke tests*):** verificação de que cada serviço arranca corretamente, responde nas portas esperadas e realiza a sua função básica.
- **Testes de integração:** verificação de que os serviços interagem corretamente entre si (por exemplo, que a VPN utiliza o Pi-hole como DNS, e que o Prometheus recolhe métricas do Node Exporter).

5.2 Teste 1: Servidor DNS (Pi-hole)

Descrição: Verificar se o Pi-hole responde a pedidos DNS na porta 53 e resolve corretamente nomes de domínio externos e internos.

Resultado esperado: Resolução bem-sucedida de `google.com` e latência inferior à do DNS do *router* (graças ao *cache* local).

Procedimento:

```
# Test DNS resolution from another device on the network
nslookup google.com 192.168.1.252
dig @192.168.1.252 google.com

# Check container status and logs
docker logs dns_pihole --tail 20
```

Resultado obtido: O Pi-hole respondeu corretamente. O painel de administração web (acessível em `http://192.168.1.252:8080/admin`) confirmou os pedidos DNS a ser processados e registados em tempo real.

Discussão: O problema de palavra-passe descrito na Secção 4.4.2 foi o único obstáculo encontrado neste serviço. Após a sua resolução, o Pi-hole manteve-se estável. Uma melhoria futura seria configurar entradas DNS locais (registos A personalizados) para os serviços internos, permitindo acesso por nome (e.g., `grafana.datacenter.local`).

5.3 Teste 2: Servidor Web (Nginx)

Descrição: Verificar se a página HTML estática é servida corretamente pelo Nginx através do volume *read-only*.

Resultado esperado: Acesso à página de estado da infraestrutura a partir de qualquer dispositivo da rede local.

Procedimento:

```
# Test with curl
curl -I http://192.168.1.252

# Check response headers
curl -v http://192.168.1.252 2>&1 | grep "< HTTP"
```

Resultado obtido: O Nginx respondeu com HTTP 200 OK. A página foi acessível a partir de um browser na rede local, confirmando o correto mapeamento do volume.

Verificação do modo read-only:

```
# Tentar escrever dentro do container (deve falhar)
docker exec nginx_proxy touch /usr/share/nginx/html/teste.txt
# Resultado esperado: touch: cannot touch '...': Read-only file
  system
```

Resultado obtido: O sistema de ficheiros dentro do *container* é efetivamente *read-only*, confirmando a aplicação correta da medida de segurança.

5.4 Teste 3: VPN (WireGuard)

Descrição: Verificar se um cliente externo (telemóvel em rede 4G) consegue estabelecer a ligação VPN e aceder aos serviços da rede interna.

Resultado esperado: Ligação VPN estabelecida, acesso ao painel Pi-hole e ao Grafana a partir da rede 4G, com todo o tráfego encriptado.

Procedimento e Resultado: O processo de diagnóstico e resolução do problema do WireGuard foi descrito em detalhe na Secção 4.4.4. Após ativação do modo DMZ no *router* e reinício físico, a ligação VPN foi estabelecida com sucesso.

Verificação da encriptação e do roteamento DNS:

Com a VPN ativa no telemóvel, verificou-se:

- O endereço IP público mostrado por <https://ip.me> mudou para o IP do servidor

(confirmando que o tráfego passa pelo túnel).

- A resolução DNS passou a ser feita pelo Pi-hole (PEERDNS=192.168.1.252), visível no painel de consultas DNS do Pi-hole.
- Os serviços internos (Grafana em 192.168.1.252:3000) ficaram acessíveis a partir do telemóvel em rede 4G.

5.5 Teste 4: *Pipeline* CI/CD

Descrição: Verificar se uma alteração ao repositório (commit) desencadeia automaticamente o re-*deploy* da infraestrutura sem intervenção humana.

Resultado esperado: *Pipeline* executada com sucesso (estado *passed*) após cada *commit* na *branch* principal.

Procedimento:

1. Alterar o ficheiro `web/index.html` (modificar o texto de estado).
2. Fazer *commit* e *push* para a *branch* principal.
3. Observar a *pipeline* na interface GitLab.

Resultado obtido: A *pipeline* foi automaticamente acionada, o *Playbook* Ansible executado e o *container* Nginx atualizado. A página web no browser refletiu a alteração após o *deploy*. A Figura 4.8 comprova o estado final da *pipeline*.

5.6 Teste 5: Monitorização (Prometheus + Grafana)

Descrição: Verificar se o Prometheus recolhe métricas do Node Exporter e se o Grafana as visualiza corretamente.

Resultado esperado: *Targets* Prometheus no estado *UP*; painel Grafana a exibir métricas de CPU, RAM e rede em tempo real.

Procedimento:


```
# Verificar targets do Prometheus
curl http://192.168.1.252:9090/api/v1/targets | python3 -m json .
    tool | grep health
```

Resultado obtido: Ambos os *targets* (*prometheus* e *node-exporter*) apresentaram o estado "health": "up". O painel *Node Exporter Full* no Grafana exibiu corretamente os gráficos de utilização de CPU, memória RAM disponível, I/O de disco e tráfego de rede.

5.7 Análise Crítica dos Resultados

5.7.1 Objetivos Cumpridos

- Todos os serviços definidos (DNS, Web, VPN, Monitorização) foram implementados e validados.
- A infraestrutura é totalmente reproduzível a partir do repositório Git.
- A *pipeline* CI/CD automatiza o *deploy* com sucesso.
- As medidas de segurança (*read-only*, isolamento de rede, WireGuard) foram aplicadas e verificadas.

5.7.2 O que Poderia ter sido Feito de Forma Diferente

- **Gestão de segredos:** As palavras-passe (Pi-hole, Grafana) estão definidas em texto simples no `docker-compose.yml`, o que não é adequado para ambientes de produção. Uma melhoria seria utilizar o **Ansible Vault** para cifrar os segredos no repositório, ou um sistema dedicado como o **HashiCorp Vault**.
- **HTTPS com certificado real:** O Nginx está configurado para HTTP na porta 80. A implementação de HTTPS com certificados *Let's Encrypt* (via **Certbot** ou

Traefik) seria o próximo passo natural para um ambiente de produção [37].

- **Centralização de *logs*:** A *stack* atual não inclui centralização de *logs* dos *containers*. A adição do **Loki** (ou da *stack* ELK) permitiria pesquisa e correlação de eventos entre serviços.

5.7.3 Objetivos que Ficaram por Cumprir

- **Servidor de E-mail:** Não foi implementado um servidor de e-mail. A complexidade de configuração de um servidor SMTP/IMAP seguro (reputação de IP, registos SPF/DKIM/DMARC, gestão de certificados) excedeu o âmbito temporal do projeto. Uma opção futura seria o **Mailcow** ou o **docker-mailserver**.
- **Servidor de Ficheiros:** Não foi implementado um servidor de ficheiros (e.g., **Nextcloud** ou **Samba**). Este serviço ficou como trabalho futuro.
- **Alertas automáticos no Grafana:** Foram configurados painéis de visualização, mas não regras de alerta (e.g., notificação por e-mail quando a utilização de CPU ultrapassa 80%).

Capítulo 6

Conclusões

6.1 Síntese do Trabalho Realizado

Este projeto teve como objetivo central demonstrar que é possível gerir uma infraestrutura de serviços de rede de forma totalmente automatizada, reproduzível e segura, utilizando exclusivamente ferramentas *open source*. O resultado final é uma plataforma funcional que corre numa máquina virtual Ubuntu Server 24.04 LTS, orquestrando múltiplos serviços em *containers* Docker: um servidor DNS (Pi-hole), um servidor Web e *reverse proxy* (Nginx), uma VPN (WireGuard) e uma *stack* de observabilidade (Prometheus, Node Exporter e Grafana).

Do ponto de vista técnico, o projeto atingiu os seus objetivos principais. A infraestrutura é definida integralmente em código (ficheiros YAML do Docker Compose e *Playbooks* Ansible), versionada no GitLab e implantada automaticamente a cada *commit* pela *pipeline* de CI/CD. Qualquer falha catastrófica no servidor poderia ser recuperada em poucos minutos, bastando provisionar uma nova VM e executar os *Playbooks* Ansible — uma garantia que, em infraestruturas geridas manualmente, simplesmente não existe.

6.2 Aprendizagens e Contribuições

O processo de implementação revelou que os maiores desafios técnicos raramente surgem da configuração dos serviços em si, mas sim das interações entre camadas distintas da infraestrutura. Os três problemas mais significativos — o *bridging* incorreto do VMware (Secção 4.2.3), os erros de indentação YAML no Netplan (Secção 4.2.2) e o bloqueio do tráfego WireGuard pelo *router* (Secção 4.4.4) — ilustram bem esta realidade: cada um exigiu um raciocínio estruturado de diagnóstico em camadas, eliminando hipóteses uma a uma até identificar a causa raiz.

A utilização do `tcpdump` para diagnosticar o problema do WireGuard foi particularmente elucidativa: ao constatar que zero pacotes chegavam à interface de rede do servidor, foi possível descartar imediatamente qualquer problema na configuração do *container* ou do protocolo, e focar o diagnóstico na camada de rede externa (o *router*). Esta abordagem metódica — observar antes de agir — é um princípio fundamental da administração de sistemas que este projeto ajudou a consolidar.

A integração do Ansible com a *pipeline* GitLab CI/CD demonstrou, de forma prática, como a filosofia de *Infrastructure as Code* elimina a dicotomia entre o que está documentado e o que está realmente instalado [1]. O repositório Git é, simultaneamente, a documentação e o estado real da infraestrutura.

6.3 Comparação com Trabalhos Relacionados

A abordagem adotada neste projeto partilha a filosofia de orquestração leve com outras soluções documentadas na literatura e na comunidade técnica. Em comparação com arquiteturas baseadas em Kubernetes [2], a solução Docker Compose apresenta menor complexidade operacional e é mais adequada para ambientes de pequena escala, como o laboratório académico. Para contextos com dezenas ou centenas de serviços e requisitos de *auto-scaling*, o Kubernetes seria a escolha natural — mas introduz uma curva de aprendizagem e uma sobrecarga de gestão que não se justificam neste cenário.

A escolha do WireGuard em detrimento do OpenVPN, fundamentada nas propriedades criptográficas e na reduzida base de código apresentadas por Donenfeld [22], revelou-se acertada: a configuração foi significativamente mais simples e o desempenho do túnel visivelmente superior em testes subjetivos de latência.

6.4 Trabalho Futuro

Com base na análise crítica dos resultados apresentada no Capítulo 5, identificam-se as seguintes linhas de trabalho futuro:

- **HTTPS com Let's Encrypt:** Implementar terminação TLS no Nginx utilizando certificados gratuitos do *Let's Encrypt*, via **certbot** ou integração com **Traefik** como *reverse proxy* alternativo com gestão automática de certificados [37].
- **Gestão de segredos com Ansible Vault:** Migrar as palavras-passe atualmente em texto simples no `docker-compose.yml` para o sistema de encriptação **Ansible Vault**, garantindo que segredos nunca são armazenados em claro no repositório [13].
- **Centralização de logs com Loki:** Adicionar o **Grafana Loki** à *stack* de observabilidade para centralizar os *logs* de todos os *containers*, complementando as métricas do Prometheus com informação de eventos [29].
- **Servidor de e-mail:** Implementar um servidor de e-mail completo (SMTP/IMAP) com o **docker-mailserver** ou **Mailcow**, incluindo os registos DNS necessários (SPF, DKIM, DMARC) para garantir a entregabilidade.
- **Servidor de ficheiros com Nextcloud:** Implementar uma instância **Nextcloud** em *container* para disponibilizar partilha de ficheiros e colaboração, completando o conjunto de serviços originalmente proposto.
- **Alertas automáticos:** Configurar regras de alerta no Grafana (utilização de CPU superior a 85%, memória RAM disponível inferior a 500 MB, *targets* Prometheus em estado *down*) com notificações por e-mail ou Telegram [29].

- **Monitorização de segurança com Fail2ban:** Integrar o **Fail2ban** para detetar e bloquear automaticamente tentativas de acesso SSH por força bruta, complementando as medidas de *hardening* já implementadas [11].

6.5 Considerações Finais

Este projeto demonstrou que a gestão profissional de infraestruturas de rede já não é exclusiva de grandes organizações com equipas dedicadas. As ferramentas *open source* disponíveis atualmente — Docker, Ansible, GitLab, Prometheus, WireGuard — são suficientemente maduras e bem documentadas para permitir que um único administrador de sistemas, com os conhecimentos técnicos adequados, construa e opere uma plataforma de serviços de rede robusta, segura e totalmente automatizada. O principal obstáculo não é tecnológico, mas metodológico: adotar desde o início a disciplina de versionamento, configuração declarativa e documentação sistemática dos problemas encontrados.

Bibliografia

- [1] K. Morris, *Infrastructure as Code: Managing Servers in the Cloud*. O'Reilly Media, 2016, ISBN: 978-1-491-92435-8.
- [2] Docker Inc. «Docker Documentation,» acedido em 1 de mar. de 2026. URL: <https://docs.docker.com/>
- [3] J. Humble e D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley Professional, 2010, ISBN: 978-0-321-60191-9.
- [4] VMware Inc. «VMware Workstation Pro Documentation,» acedido em 28 de fev. de 2026. URL: <https://docs.vmware.com/en/VMware-Workstation-Pro/index.html>
- [5] IBM. «Containerization Explained,» acedido em 1 de mar. de 2026. URL: <https://www.ibm.com/topics/containerization>
- [6] DigitalOcean. «How To Install and Use Docker on Ubuntu 22.04,» acedido em 28 de fev. de 2026. URL: <https://www.digitalocean.com/community/tutorials/how-to-install-and-use-docker-on-ubuntu-22-04>
- [7] Fireship. «Docker in 100 Seconds.» Video YouTube, acedido em 28 de fev. de 2026. URL: <https://www.youtube.com/watch?v=Gjnup-PuquQ>
- [8] Docker Inc. «Docker Compose Overview,» acedido em 1 de mar. de 2026. URL: <https://docs.docker.com/compose/>

- [9] DigitalOcean. «How To Install and Use Docker Compose on Ubuntu,» acedido em 1 de mar. de 2026. URL: <https://www.digitalocean.com/community/tutorials/how-to-install-and-use-docker-compose-on-ubuntu-20-04>
- [10] Docker Inc. «Docker Security Best Practices,» acedido em 21 de abr. de 2026. URL: <https://docs.docker.com/engine/security/>
- [11] M. Souppaya, J. Morello e K. Scarfone, «Application Container Security Guide,» National Institute of Standards e Technology (NIST), rel. téc. NIST SP 800-190, 2017. acedido em 22 de abr. de 2026. URL: <https://doi.org/10.6028/NIST.SP.800-190>
- [12] Red Hat. «Podman: A Tool for Managing OCI Containers and Pods,» acedido em 15 de mar. de 2026. URL: <https://podman.io/>
- [13] Red Hat. «Ansible Documentation,» acedido em 28 de fev. de 2026. URL: <https://docs.ansible.com/>
- [14] Red Hat. «Ansible Concepts: Idempotency,» acedido em 28 de fev. de 2026. URL: https://docs.ansible.com/ansible/latest/getting_started/basic_concepts.html
- [15] TechWorld with Nana. «Ansible Tutorial for Beginners: Ultimate Playbook and Ad Hoc Commands.» Video YouTube, acedido em 28 de fev. de 2026. URL: <https://www.youtube.com/watch?v=1id6ERvfozo>
- [16] P. Mockapetris. «Domain Names — Implementation and Specification (RFC 1035),» Internet Engineering Task Force (IETF), acedido em 1 de mar. de 2026. URL: <https://www.rfc-editor.org/rfc/rfc1035>
- [17] Pi-hole LLC. «Pi-hole — Network-Wide Ad Blocking,» acedido em 1 de mar. de 2026. URL: <https://pi-hole.net/>
- [18] Pi-hole LLC. «Pi-hole Docker Documentation,» acedido em 1 de mar. de 2026. URL: <https://github.com/pi-hole/docker-pi-hole>

- [19] NetworkChuck. «Pi-Hole on Docker – the BEST way to run Pi-Hole!» Video YouTube, acessado em 1 de mar. de 2026. URL: <https://www.youtube.com/watch?v=NRe2-vye3ik>
- [20] F5 Networks / Nginx Inc. «Nginx Documentation,» acessado em 1 de mar. de 2026. URL: <https://nginx.org/en/docs/>
- [21] NGINX Inc. «What Is a Reverse Proxy Server?» Acessado em 21 de abr. de 2026. URL: <https://www.nginx.com/resources/glossary/reverse-proxy-server/>
- [22] J. A. Donenfeld, «WireGuard: Next Generation Kernel Network Tunnel,» 2017. acessado em 30 de abr. de 2026. URL: <https://www.ndss-symposium.org/ndss2017/ndss-2017-programme/wireguard-next-generation-kernel-network-tunnel/>
- [23] NetworkChuck. «WireGuard: How to Create a VPN Server in 5 Minutes.» Video YouTube, acessado em 30 de abr. de 2026. URL: <https://www.youtube.com/watch?v=bVKNSf1p1d0>
- [24] LinuxServer.io. «LinuxServer WireGuard Docker Image Documentation,» acessado em 30 de abr. de 2026. URL: <https://docs.linuxserver.io/images/docker-wireguard/>
- [25] B. Beyer, C. Jones, J. Petoff e N. R. Murphy, *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media, 2016, ISBN: 978-1-491-92912-4. URL: <https://sre.google/sre-book/table-of-contents/>
- [26] TechWorld with Nana. «Prometheus Tutorial: Understand Prometheus in 6 Hours.» Video YouTube, acessado em 22 de abr. de 2026. URL: <https://www.youtube.com/watch?v=h4S121AKiDg>
- [27] Cloud Native Computing Foundation. «Prometheus Documentation,» acessado em 22 de abr. de 2026. URL: <https://prometheus.io/docs/introduction/overview/>
- [28] Cloud Native Computing Foundation. «Prometheus Node Exporter,» acessado em 22 de abr. de 2026. URL: https://github.com/prometheus/node_exporter

- [29] Grafana Labs. «Grafana Documentation,» acedido em 22 de abr. de 2026. URL: <https://grafana.com/docs/grafana/latest/>
- [30] Grafana Labs. «Node Exporter Full — Grafana Dashboard 1860,» acedido em 22 de abr. de 2026. URL: <https://grafana.com/grafana/dashboards/1860-node-exporter-full/>
- [31] GitLab Inc. «GitLab CI/CD Documentation,» acedido em 21 de abr. de 2026. URL: <https://docs.gitlab.com/ee/ci/>
- [32] GitLab Inc. «GitLab Runner Documentation,» acedido em 21 de abr. de 2026. URL: <https://docs.gitlab.com/runner/>
- [33] TechWorld with Nana. «GitLab CI CD Pipeline Tutorial Including Deployment to a Server.» Video YouTube, acedido em 21 de abr. de 2026. URL: <https://www.youtube.com/watch?v=qP8kir2GUgo>
- [34] Canonical Ltd. «Ubuntu Server Documentation,» acedido em 28 de fev. de 2026. URL: <https://ubuntu.com/server/docs>
- [35] Canonical Ltd. «Netplan Documentation,» acedido em 28 de fev. de 2026. URL: <https://netplan.readthedocs.io/en/stable/>
- [36] Microsoft. «Windows Subsystem for Linux Documentation,» acedido em 28 de fev. de 2026. URL: <https://learn.microsoft.com/en-us/windows/wsl/>
- [37] E. Rescorla. «The Transport Layer Security (TLS) Protocol Version 1.3 (RFC 8446),» Internet Engineering Task Force (IETF), acedido em 29 de abr. de 2026. URL: <https://www.rfc-editor.org/rfc/rfc8446>

Apêndice A

Proposta Original do Projeto



Curso de Licenciatura em Engenharia Informática
Projeto 3º Ano - Ano letivo de 2016/2017

<Título do projeto>

Orientador: <Nome do orientador>

Coorientador: <Nome do coorientador>

1 Objetivo

<Objetivo do projeto>

2 Detalhes

<Detalhes que julguem ser necessários>

3 Metodologia de trabalho

<Eventual metodologia de trabalho>

Dimensão da equipa:

Recursos necessários:

Apêndice B

Outro(s) Apêndice(s)

Listagens de código fonte, texto/imagens produzidos por testes complementares, etc.